MakeMistakes — Build Phase Strategy

Recommended approach for Phase 5: training students to build products rather than simply receiving AI-generated code.

1. Core Idea

The current direction is to use the large Problem Library as a progression system. Students should not simply receive complete implementation code from AI. The platform should train them to actually build the product, with increasing engineering responsibility as problem difficulty increases.

2. Difficulty Progression

Level	Recommended Build Experience
Starter	1 focused implementation task. Student follows clear requirements and writes the implementation.
Foundation	2 connected tasks. The second task depends on the first, introducing basic system thinking.
Intermediate	2–3 interconnected tasks involving APIs, databases, authentication, integration, or similar engineering decisions.
Advanced	A system-level objective broken into milestones. Students must make architecture and trade-off decisions.
Expert	An ambiguous real-world problem with constraints and expected outcomes, but without a step-by-step recipe.

3. Do Not Measure Difficulty Only by Task Count

A simple rule such as Starter = 1 task, Intermediate = 3 tasks, Advanced = 5 tasks measures quantity rather than engineering ability. A student could complete several trivial tasks and learn less than another student who completes one genuinely difficult engineering task.

Instead, difficulty should increase the amount of engineering responsibility placed on the student.

4. Recommended Responsibility Progression

STARTER → Follow requirements
FOUNDATION → Implement connected features
INTERMEDIATE → Make engineering decisions
ADVANCED → Design systems and handle trade-offs
EXPERT → Solve ambiguous real-world problems

5. Examples

Starter

Example: Create the reminder creation form. MakeMistakes provides requirements, expected behavior, constraints, hints, documentation, and validation, but the student writes the implementation.

Foundation

Example: Task 1 — Create a reminder. Task 2 — Store and retrieve the reminder. The student begins thinking about how components connect.

Intermediate

Example: Task 1 — Authentication → Task 2 — API → Task 3 — Database integration. The student builds a small connected system rather than isolated features.

Advanced

Example objective: Design a notification system that supports multiple users, retries failed notifications, handles timezone differences, and remains reliable when the network is unavailable. The student works through architecture, milestones, integration, and testing.

Expert

The platform gives the student a problem, requirements, constraints, and an expected outcome, but does not provide a recipe. The student researches, chooses an architecture, builds, tests, and explains the engineering decisions.

6. Role of AI

AI should act as a mentor rather than a code vending machine. Instead of immediately supplying corrected code, it should help the student reason through problems.

Example: Instead of saying “Here is the corrected code,” the mentor could ask: “Your API returns 500 when the database is unavailable. What do you think should happen?”

7. Recommended Build Data Model

Do not hard-code a fixed number of tasks per difficulty level. Store build requirements with each problem so the Build Engine can dynamically control the experience.

```
problem
■■■ difficulty
■■■ buildObjective
■■■ buildTasks[]
■■■ constraints[]
■■■ hints[]
■■■ validationCriteria[]
■■■ expectedOutcome
```

8. Integration With the Existing 8-Phase Workflow

The Build Phase strategy should not require redesigning the existing eight phases. The current workflow remains:

1. Discover
2. Research
3. Design
4. Plan
5. Build
6. Test
7. Deploy
8. Improve

Phase 5 becomes progressively more demanding according to the problem's difficulty and the student's demonstrated ability.

9. Final Recommendation

The strongest version of MakeMistakes should measure difficulty by responsibility, ambiguity, decision-making, system complexity, and engineering trade-offs — not merely by the number of tasks. This creates a progression from following instructions to independently solving real-world engineering problems.